

[Marcar como hecha](#)

12.4. Utilización de JavaBeans

EJBs

Un EJB es un componente del lado del servidor que encapsula la lógica del negocio de una aplicación. En cualquier aplicación, los beans enterprise implementan los métodos de la lógica del negocio, que pueden ser invocados por clientes remotos para acceder a los servicios importantes proporcionados por la aplicación.

Beneficios

Los EJBs simplifican el desarrollo de grandes aplicaciones empresariales seguras y distribuidas por las siguientes razones:

Los desarrolladores pueden concentrarse en solventar la lógica del negocio: el contenedor EJB proporciona servicios a nivel del sistema como el control de transacciones y las autorizaciones de seguridad. Por lo tanto, los desarrolladores no tienen que preocuparse de estos problemas.

Clientes pequeños: Los desarrolladores no tienen que desarrollar código para las reglas de negocio o accesos a bases de datos; pueden concentrarse en la presentación del cliente. El resultado final es que los clientes son pequeños, y esto es especialmente importante para clientes que se ejecutan en pequeños dispositivos con recursos limitados.

Desarrollo rápido: Los EJBs son componentes portables, y por lo tanto los ensambladores de aplicaciones pueden construir nuevas aplicaciones desde beans existentes. Las aplicaciones resultantes se pueden ejecutar en cualquier servidor compatible J2EE.

Componentes

Hay dos tipos principales de componentes EJB : session y entity. Un EJB de sesión se usa para realizar una tarea para un cliente, y un EJB de entidad es específico del dominio y se usa para representar un objeto de entidad del negocio que existe en un almacenamiento persistente. Sin embargo, los beans de entidad y de sesión tienen algunas diferencias que podemos ver en la siguiente tabla:

EJB de Sesión

- Transitorio
- Puede ser usado por un sólo cliente.
- No tiene identidad

EJB de Entidad

- Persistente
- Puede ser usado por muchos clientes.
- Tiene una identidad (como una clave primaria)

Desarrollar EJBs

Para desarrollar EJBs, todo bean enterprise necesita:

- Un **interface remoto** que exponga los métodos que soporta bean enterprise.
- Un **interface home** que proporciona los métodos del ciclo de vida del bean enterprise.
- Una clase de implementación, incluyendo la lógica de negocio que necesite.

EJBs contra Servlets

A primera vista, los EJBs y los Servlets son tecnologías similares porque ambos son

?

componentes distribuidos del lado del servidor. Sin embargo, hay una diferencia importante entre los dos en el tipo de solución que ofrecen; los EJBs no pueden aceptar peticiones HTTP. En otras palabras, los EJBs no pueden servir peticiones que vienen directamente desde un navegador Web, y los servlets sí pueden. Servlets y JSPs se pueden usar para implementar presentación y control web, pero no pueden manejar transacciones distribuidas, al contrario que los EJBs. Estos pueden ser llamados desde cualquier cliente basado en Java.

¿Cuándo usar EJBs?

Los EJBs son buenos para las aplicaciones que tienen alguno de estos requerimientos:

Escalabilidad: si tenemos un número creciente de usuarios, los EJBs nos permitirán distribuir los componentes de nuestra aplicación entre varias máquinas con su localización transparente para los usuarios.

Integridad de Datos: los EJBs nos facilitan el uso de transacciones distribuidas.

Variedad de clientes: si nuestra aplicación va a ser accedida por una variedad de clientes (como navegadores tradicionales o navegadores WAP), se pueden usar los EJBs para almacenar el modelo del negocio, y se puede usar una variedad de clientes para acceder a la misma información.

Ejemplos

Elementos de Script

En el ejemplo date.jsp se usa todo el nombre de la clase Date incluyendo el nombre del paquete, lo que podría llegar a ser tedioso. Si queremos crear un ejemplar de la clase Date usando simplemente: Date today = new Date(); sin tener que especificar el path completo de la clase, usamos la directiva page de esta forma:

Ejemplo #1: fecha.jsp

```
<%@page import="java.util.*" %>
```

```
<HTML>
```

```
<HEAD>
```

```
<TITLE>JSP Example</TITLE>
```

```
</HEAD>
```

```
<BODY BGCOLOR="ffffcc">
```

```
<CENTER>
```

```
<H2>Date and Time</H2>
```

```
<%
```

```
java.util.Date today = new java.util.Date();
```

```
out.println("Today's date is: "+today);
```

```
%>
```

```
</CENTER>
```

```
</BODY>
```

```
</HTML>
```

Todavía hay otra forma de hacer lo mismo usando la etiqueta <p> escribiendo: Today's date is: <%= new Date() %> </p>

Como podemos ver, se puede conseguir el mismo resultado usando diferentes etiquetas y técnicas. Hay varios elementos de script JSP. Hay algunas reglas convencionales que nos ayudarán a usar más efectivamente los elementos de Script JSP.

1- Usamos <% ... %> para manejar declaraciones, expresiones, o cualquier otro tipo de código válido.

2- Usamos la directiva page como en <%@page ... %> para definir el lenguaje de script. También puede usarse para especificar sentencias import. Aquí hay un ejemplo:

```
<%@page language="java" import="java.util.*" %>
```

3- Usamos <%! %> para declarar variables o métodos. Por ejemplo:

```
<%! int x = 10; double y = 2.0; %>
```

4- Usamos <%= ... %> para definir una expresión y forzar el resultado a un String. Por ejemplo: <%= a+b %> o <%= new java.util.Date() %>.

5- Usamos la directiva include como en <%@ include ... %> para insertar el contenido de otro fichero en el fichero JSP principal. Por ejemplo:

?

```
<%@include file="copyright.html" %>
```

Manejar Formularios

Una de las partes más comunes en aplicaciones de web es un formulario HTML donde el usuario introduce alguna información como su nombre y dirección. Usando JSP, los datos del formulario (la información que el usuario introduce en él) se almacenan en un objeto request que es enviado desde el navegador hasta el contenedor JSP. La petición es procesada y el resultado se envía a través de un objeto response de vuelta al navegador. Estos dos objetos están disponibles implícitamente para nosotros.

Para demostrar como manejar formularios HTML usando JSP, aquí tenemos un formulario de ejemplo con dos campos: uno para el nombre y otro para el email. Como podemos ver, el formulario HTML está definido en un fichero fuente JSP. Se utiliza el método request.getParameter para recuperar los datos desde el formulario en variables creadas usando etiquetas JSP.

La página procesar.jsp imprime un formulario o la información proporcionada por el usuario dependiendo de los valores de los campo del formulario. Si los valores del formulario son null se muestra el formulario, si no es así, se mostrará la información proporcionada por el usuario. Observa que el formulario es creado y manejado por el código del mismo fichero JSP.

Ejemplo #2: procesar.jsp

```
<HTML>
<HEAD>
  <TITLE>Formulario</HEAD>
Ejemplo</TITLE>
<BODY BGCOLOR="#ffffcc">
  <% if (request.getParameter("name")==null &&
    request.getParameter("email")== null) { %>
    <CENTER>
    <H2>User Info Request Form</H2>
    <FORM METHOD="GET" ACTION="procesar.jsp">
    <P>Nombre: <input type="text" name="name" size=26>
    <P>email: <input type="text" name="email" size=26>
    <P><input type="submit" value="Process">
    </FORM>
    </CENTER>
  <% } else { %>
    <%! String name, email; %>
    <% name = request.getParameter("name");
      email = request.getParameter("email"); %>
    <P><B>Ha introducido la siguiente información</B>:
    <P><B>Nombre</B>: <%= name %>
    <P><B>Email</B>: <%= email %>
  <% } %>
</BODY>
</HTML>
```

Comentarios: En el contexto de JSP, los JavaBeans contienen la lógica de negocio que devuelve datos a un script en una página JSP, que a su vez formatea los datos devueltos por el componente JavaBean para su visualización en el navegador. Una página JSP utiliza un componente JavaBean fijando y obteniendo las propiedades que proporciona.

Hay muchos beneficios en la utilización de JavaBeans para mejorar las páginas JSP:

- Componentes Reutilizables: diferentes aplicaciones pueden reutilizar los mismos componentes.
- Separación de la lógica de negocio de la lógica de presentación: podemos modificar la forma de mostrar los datos sin que afecte a la lógica del negocio.
- Protegemos nuestra propiedad intelectual manteniendo secreto nuestro código fuente.

?

Uso de JavaBeans con JSP

Ahora, veamos como modificar el ejemplo anterior, procesar.jsp para usar JavaBeans. En el formulario anterior había dos campos: name y email. En JavaBeans, son llamados propiedades. Por eso, primero escribimos un componente JavaBean con métodos setX getX, donde X es el nombre de la propiedad. Por ejemplo, si tenemos unos métodos llamados setName y getName entonces tenemos una propiedad llamada name. El ejemplo #3 muestra un componente FormBean.

Los buenos componentes deben poder interoperar con otros componentes de diferentes vendedores. Por lo tanto, para conseguir la reutilización del componente, debemos seguir dos reglas importantes (que son impuestas por la arquitectura JavaBeans):

1.Nuestra clase bean debe proporcionar un constructor sin argumentos para que pueda ser creado usando Beans.instantiate.

2.Nuestra clase bean debe soportar persistencia implementando el interface Serializable o Externalizable.

Ejemplo #3: FormBean.java

```
package userinfo;

import java.io.*;

public class FormBean implements Serializable {
    private String name;
    private String email;
    public FormBean() {
        name = null;
        email = null;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getEmail() {
        return email;
    }
}
```

Para poder usar el componente FormBean en el fichero JSP, necesitamos ejemplarizar el componente. Esto se hace usando la etiqueta <jsp:useBean>. La siguiente línea <jsp:setProperty> se ejecuta cuando se ha ejemplarizado el bean, y se usa para inicializar sus propiedades. En este caso, ambas propiedades (name y email) se configuran usando una sola sentencia. Otra posible forma de configurar las propiedades es hacerlo una a una, pero primero necesitamos recuperar los datos desde el formulario. Aquí tenemos un ejemplo de como configurar la propiedad name:

```
<%! String yourname, youremail; %>
<% yourname = request.getParameter("name"); %>
<jsp:setProperty name="formbean" property="name"
value="<%=yourname%>" />
```

Una vez que se han inicializado las propiedades con los datos recuperados del formulario, se recuperan los valores de las propiedades usando <jsp:getProperty> en la parte else, como se ve en el Ejemplo #4:

Ejemplo #4: procesar2.jsp

```
<jsp:useBean id="formbean" class="userinfo.FormBean" />
<jsp:setProperty name="formbean" property="*" />
<HTML>
<HEAD>
```

?

```
<TITLE>Form Example</TITLE>
</HEAD>
<BODY BGCOLOR="#ffffcc">
<% if (request.getParameter("name")==null
&& request.getParameter("email") == null) { %>
<CENTER>
<H2>User Info Request Form </H2>
<form method="GET" action="procesar2.jsp">
<P>
Your name: <input type="text" name="name" size=27>
<p>
Your email: <input type="text" name="email" size=27>
<P>
<input type="submit" value="Process">
</FORM>
</CENTER>
<% } else { %>
<P>
<B>You have provided the following info</B>:
<P>
<B>Name</B>: <jsp:getProperty name="formbean" property="name"/>
<P>
<B>Email</B>: <jsp:getProperty name="formbean" property="email"/>
<% } %>
</BODY>
</HTML>
```

Última modificación: miércoles, 15 de marzo de 2023, 10:44